Can AI Find What Developers Need? Defining and Measuring Knowledge Availability for AI in Developer Ecosystems

ANONYMOUS AUTHOR(S)

Developers who delegate technical searches to AI agents still need to judge whether suggestions have supporting evidence, fit their local environment, and leave important information unresolved. These judgments depend in part on the knowledge agents can obtain. We define knowledge availability for AI and operationalize it through a task-level protocol with eleven indicators covering sources, acquisition effort, and response properties. We apply the method to 26 development tasks: 25 CANN/CUDA pairs and one separate migration analogy. The analysis distinguishes missing operational content from accessible but insufficient guidance, identifies recurring version issues, and reveals different combinations of knowledge support. These distinctions specify information needed to assess a proposed technical action and locate potential improvements in knowledge provision and agent feedback. We contribute a framework, an inspectable protocol, and a case analysis of the knowledge conditions underlying developers' access to technical guidance through AI.

CCS Concepts: • **Human-centered computing** → **HCI theory, concepts and models**; • **Software and its engineering** → *Documentation.*

Additional Key Words and Phrases: knowledge availability for AI; developer ecosystems; technical documentation; information foraging; AI agents; measurement methods

1 INTRODUCTION

Developers routinely move between documentation, code examples, search results, and community explanations while deciding what to do next. Research on opportunistic programming and developers' information needs shows that finding and interpreting information is part of programming itself [2, 6, 12]. A useful result must fit the current task: a command needs the right options, an API example needs the right version, and an explanation of an error needs enough context to guide investigation. The existence of documentation is therefore only one condition for its usefulness.

Development agents with retrieval and tool-use capabilities introduce another collaborative route through this knowledge environment. A developer may ask about an error or a code fragment, or state a desired change or outcome and delegate information seeking, documentation reading, implementation planning, and portions of code modification to an agent. To advance such a task, the agent may search, read, and combine material across official sites, documentation, repositories, and communities before drafting a response, an implementation approach, or a code change. Studies of code-generation tools, conversational programming assistance, and agent support describe opportunities for this support alongside difficulties in understanding and checking generated suggestions [1, 10, 13]. Delegating the search changes how supporting material reaches the developer, while leaving a need to judge whether a suggestion has an identifiable basis, fits the local environment, and requires further information. The agent's access to technical material is one condition for making that basis available for inspection. Measuring this access can therefore identify gaps in the knowledge on which AI-mediated guidance depends.

Consider an agent asked how to implement and integrate a custom accelerator operator. A search may return an official development guide, yet the reading tool may obtain only navigation and metadata. For an error-diagnosis task, the same tool may obtain the full official page, but the page may contain only a generic instruction to inspect logs. A retrieval-success measure can mark both questions as having relevant results. An access measure distinguishes the

2026. Manuscript submitted to ACM

first failure but not the second. Even adequate instructions can remain ambiguous when they refer to incompatible toolkit and framework versions. These distinctions matter to documentation teams deciding what to repair and to agent designers deciding what uncertainty to expose.

Existing evaluation approaches already distinguish retrieval quality, context relevance, answer faithfulness, and related properties. RAGAS and ARES provide component-level evaluation of retrieval-augmented generation, while ALCE examines generated answers and their citations [3, 4, 11]. Our contribution addresses a complementary measurement problem: how to inspect the technical knowledge conditions encountered by an agent while addressing a concrete development task, including access failures, version relations, alternative sources, and the provenance of an auditor's judgments. This requires connecting the developer's information requirement to the resources actually obtained, rather than inferring availability from a fluent answer.

We define **knowledge availability for AI** as the extent to which task-relevant technical knowledge can be discovered, obtained, and assessed for applicability by a specified agent and tool configuration under stated retrieval conditions. The construct is relational: it concerns a task, a knowledge environment, and a means of accessing that environment at a particular time. We operationalize it through an evidence-recording protocol and eleven indicators. The indicators distinguish source conditions, acquisition effort, response checks, and a composite confidence score calculated from M1-M8. The intended users of the method are documentation maintainers and agent product and interface teams. It is designed to help them locate missing source material, unresolved applicability conditions, and gaps that an interface may need to communicate. The case application demonstrates diagnostic distinctions for these uses; their value in teams' work requires evaluation.

We ask two research questions. **RQ1:** How can knowledge availability for AI be defined and operationalized so that task-specific access conditions and evidence gaps can be systematically recorded and reviewed? **RQ2:** What patterns of knowledge availability does the method reveal across development tasks, and how can those patterns inform documentation and agent design?

We address these questions through a methodological framework and its application to software development tasks for AI-accelerated computing. The case covers 26 task categories, yielding 52 task-side records. Twenty-five categories compare CANN and CUDA development contexts; one migration category uses ROCm/HIP as the comparison destination and is treated separately. The case application connects task-level measurement to cross-task patterns and design implications. We contribute (1) a construct that distinguishes knowledge conditions from retrieval success and answer correctness; (2) a task-level protocol, indicator definitions, and portable analysis materials; and (3) worked cases and a cross-task synthesis that distinguish different breakdowns and connect them to documentation and agent design.

2 RELATED WORK AND CONSTRUCT BOUNDARIES

2.1 Developer information seeking and documentation

Information foraging theory relates information-seeking behavior to the structure and expected value of available information [8]. Programming studies show how developers interleave searching, learning, and implementation, and how their questions depend on the work they are performing [2, 6, 12]. API-learning difficulties further motivate attention to the explanatory resources surrounding a technical interface [9]. Taken together, these studies motivate an assessment organized around the information needed for a programming action. We use task requirements to examine whether material obtained through an agent supplies an explanation, parameter, or procedure needed for that action.

This connects the unit of measurement to developers' information needs rather than treating a returned page as the endpoint of the search.

Our method does not infer human usability from machine access. A page readable by a browser can be inaccessible to a particular extraction tool; conversely, text readily extracted by an agent can still be difficult for a person to navigate. We examine the conditions of the delegated knowledge route. Claims about developers' time, trust, satisfaction, or task completion require separate evidence.

2.2 AI support for programming

Studies of AI programming assistance distinguish obtaining a suggestion from understanding, adapting, and checking it. Vaithilingam et al. describe difficulties in understanding, editing, and debugging generated code [13]; Barke et al. distinguish acceleration and exploration in programmers' use of code-generating models [1]. Ross et al. examine conversational assistance around code context [10]. These findings motivate attention to the supporting information available when a suggestion is considered. Our protocol examines whether relevant source passages, version conditions, and procedural details can be obtained and connected to that suggestion. It supplies an account of those knowledge conditions alongside studies of how developers engage with AI assistance.

Agents can interleave reasoning and tool use, and retrieval-augmented generation provides a way to incorporate external information into generation [7, 15]. WebArena, SWE-bench, and SWE-agent evaluate or develop agents in realistic web and software-engineering settings [5, 14, 16]. Their task outcomes are valuable evidence of system performance. Our framework offers a complementary description of the knowledge conditions under which such systems operate. A failure can involve unavailable evidence, inadequate interpretation of available evidence, or unsuccessful execution; the present method directly addresses the first of these and records information relevant to separating them.

2.3 Retrieval, attribution, and the availability construct

RAGAS and ARES assess properties of retrieved context and generated answers; ALCE evaluates citation-supported generation [3, 4, 11]. Accordingly, we do not claim that previous evaluation treats retrieval or answer quality as indivisible. Our methodological focus is the connection between technical-task requirements, the acquisition process, and the structure of the surrounding knowledge ecosystem. Version compatibility, partial access to a how-to guide, and dependence on a community workaround are particularly consequential in this setting.

The distinctions in Table 1 position the construct. Availability can be necessary for an evidence-grounded answer without being sufficient for correctness. An agent can misinterpret an available source. It can also produce a correct answer from prior knowledge without obtaining any external evidence. These outcomes should remain distinguishable.

Table 1. Construct boundaries and neighboring objects of assessment.

Object of assessment	Central question	Relationship to this method
Retrieval success	Was a relevant result returned?	One acquisition condition; not proof that required content was obtained.
Documentation usability	Can intended readers navigate and understand the material?	A related human-facing property requiring its own evaluation.

Object of assessment	Central question	Relationship to this method
Knowledge availability for AI	What task-relevant knowledge was obtainable under the recorded conditions?	The construct operationalized here.
Answer correctness and attribution	Is the response correct, and do its sources support its claims?	A downstream evaluation that may use the acquired evidence.
Execution success	Do the proposed actions work in the target environment?	Requires execution or other independent outcome evidence.

3 METHODOLOGICAL FRAMEWORK

3.1 Unit of analysis and task requirements

The unit is a **task-side acquisition episode**: a development objective pursued in a particular technical ecosystem using a stated agent and retrieval configuration. A task specification records the desired outcome, starting artifacts, constraints, version dependencies, and evidence required to support a next action. For example, model conversion may require a conversion command, an input-shape specification, a target-device setting, and a way to check the generated artifact. These requirements guide inspection; a long page is not automatically adequate.

Task selection must follow the purpose of the application. An ecosystem assessment may seek coverage across workflows; a documentation-team audit may focus on recurring support questions. Neither establishes the population frequency of those tasks without additional sampling evidence. For cross-ecosystem use, analysts record differences in starting conditions and task scope rather than assuming that similar names make tasks equivalent. The framework can also be applied within one ecosystem, across versions, or across retrieval configurations, provided the comparison conditions are made explicit.

Figure 1 locates the measurements in an observable tool-use loop. It is an analytic model, not a reconstruction of hidden model reasoning: the agent searches for candidate sources, selects URLs, fetches content or records a failure, then integrates evidence and assesses task adequacy and version applicability. When evidence is insufficient but remains searchable, the loop returns to query refinement. Model prior is registered as a separate branch that need not produce an observable source; it is therefore not automatically treated as traceable evidence.

3.2 Sources, acquisition, and action conditions

The framework examines three knowledge channels: official materials, community or third-party materials, and model prior knowledge. For external channels, it separates **access** from **adequacy**. Discovery concerns whether a candidate source is found; acquisition concerns what the reading tool actually returns. Adequacy concerns whether that return supplies the task-required commands, explanations, parameters, or diagnostic cases. Version applicability is examined across channels because sources can be individually informative while describing different environments.

Official status is determined by the publisher's relationship to the material, not by its hosting platform. A vendor's repository or blog belongs to that vendor's official channel. An independently authored tutorial hosted on the same platform can have a different status. Cross-posting also makes domain diversity an imperfect proxy for independent evidence. The protocol therefore preserves source identity, publisher, date when available, and any observed derivation or duplication. Lack of these details is recorded as uncertainty.

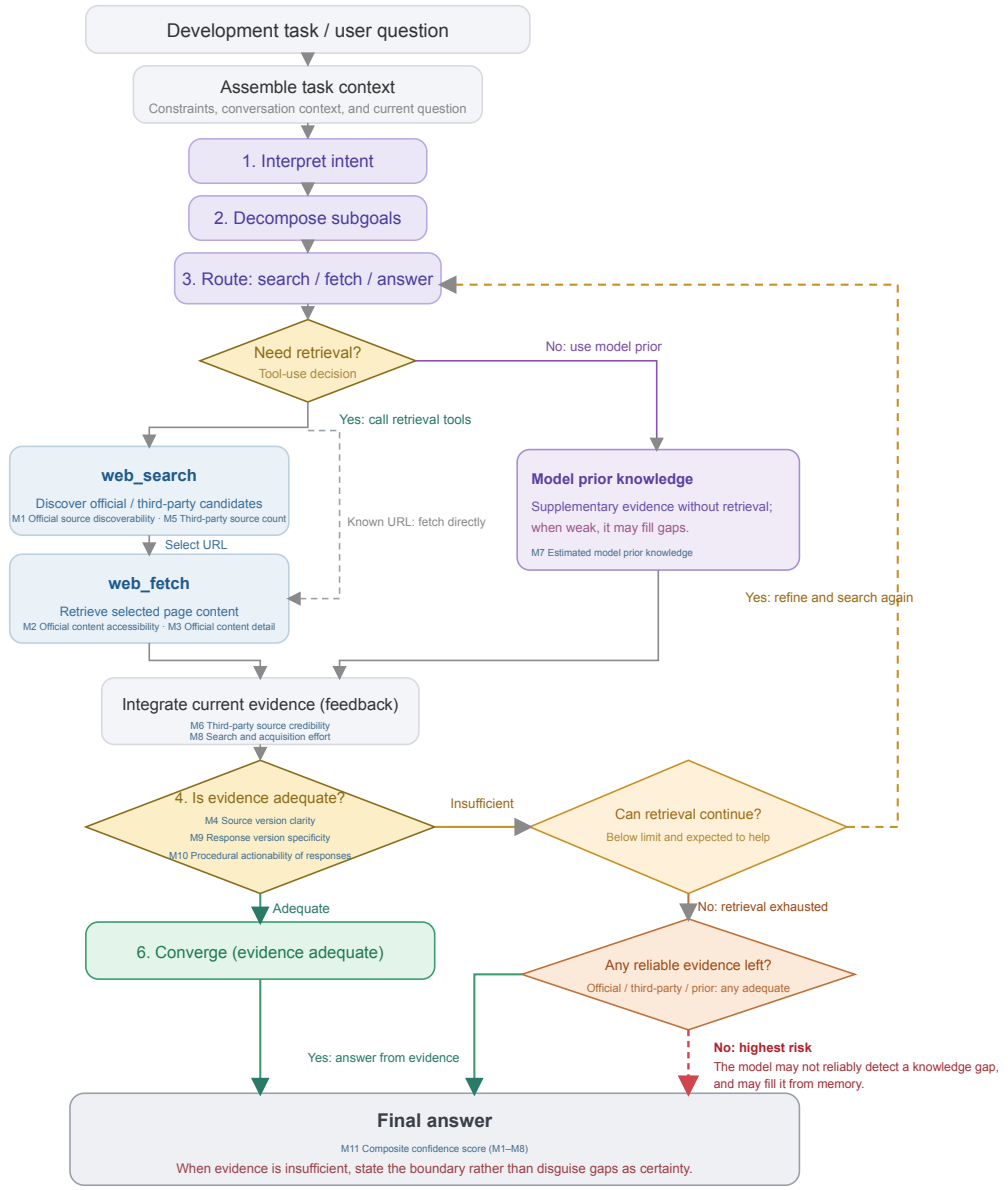

Fig. 1. An agent starts from a task and context, interprets intent, decomposes subgoals, and routes to retrieval or model prior; a retrieval branch searches, selects URLs, fetches content or fetches a known URL directly, then integrates evidence and assesses adequacy. Insufficient evidence can trigger retry; exhausted retrieval distinguishes whether reliable evidence remains.

The output of the method is an **availability profile** with an evidence trail. It identifies what was found, what was obtained, what appears adequate, what applicability conditions remain unresolved, and the effort required to acquire that knowledge. An optional aggregate can help summarize a chosen operationalization, but it cannot replace

this profile. In particular, internal model knowledge cannot make an inaccessible external source accessible, even if it enables an answer.

3.3 An evidence record

The evidence record connects each judgment to its basis. Its minimum fields are the task question and starting conditions; agent and tool configuration when known; collection time; queries and selected URLs; publisher and source role; returned content or a relevant excerpt; access outcome; applicable versions; task requirements supported by that content; acquisition counts; and the coding decision with a rationale. A field also identifies whether an entry is directly recorded, subsequently coded, estimated, or unavailable.

This distinction prevents a common reporting error: transforming an interpretation into an observation merely because a script assigns it a number. A logged extraction failure is an observation of the tool-source interaction. Labeling an explanation as sufficient is a judgment against task requirements. Estimating a model's familiarity with a toolkit is an inference unless a separate test supports it. Numerical coding does not erase these differences.

4 OPERATIONALIZATION AND MEASUREMENT PROTOCOL

4.1 Eleven indicators and their evidential roles

The method examines three knowledge sources: official material, third-party material, and model prior knowledge. It assesses discovery, acquisition, and source quality, and connects knowledge acquisition to response generation through measures of retrieval effort and response properties. Table 2 presents the eleven indicators and their scoring criteria.

Table 2. The eleven indicators and the evidence each can support.

ID	Indicator	Evidence and interpretation
M1	Official source discoverability	Measures the ease of finding task-relevant official material using result position, search rounds, and the need for query refinement.
M2	Official content accessibility	Assesses the retrieval of task-relevant official content using the selected page's fetch category, returned body content, and access barriers.
M3	Official content detail	Measures the detail of task-relevant official material using its coverage level and the actionability of commands and code. Content that cannot be retrieved is marked as blocked.
M4	Source version clarity	Measures how clearly official material enables selection of an applicable version, considering coexisting versions, compatibility matrices, and device–framework compatibility conditions.
M5	Third-party source count	Measures the amount of third-party material obtained during retrieval, with scores assigned according to the recorded source count.
M6	Third-party source credibility	Assesses third-party credibility from source type, with adjustments for publication age, content consistency, and distribution across source platforms.

ID	Indicator	Evidence and interpretation
M7	Estimated model prior knowledge	Estimates the support that existing model knowledge provides for a task, using model self-assessment adjusted for the pace of relevant technical change.
M8	Search and acquisition effort	Measures retrieval effort from search rounds, fetch attempts, and failed fetches. Lower acquisition cost receives a higher score.
M9	Response version specificity	Measures how precisely a response identifies applicable versions and component combinations, ranging from exact specifications to unresolved version choices.
M10	Procedural actionability of responses	Assesses the operational completeness of response commands and steps, distinguishing direct use, parameter substitution, minor adaptation, and skeletal guidance.
M11	Composite confidence score	Combines support from official material, third-party material, and model prior knowledge, with adjustments for version clarity and retrieval effort, using the formula below.

M1–M7 describe support from the three knowledge sources, while M8 captures the effort required to acquire that knowledge. M9 and M10 assess response version specificity and procedural actionability. M11 is calculated from M1–M8, with response properties reported separately. Together, the individual indicators and composite confidence characterize each task's knowledge support and help identify areas for improvement.

4.2 Applying the protocol

First, define the task and the recording boundary. State the development outcome, starting artifacts, and requirements against which adequacy will be assessed. Record the agent, tools, language, date, and context policy. Specify whether the unit includes a fresh acquisition, reused observations, or both. Set a retrieval budget or another observable stopping criterion appropriate to the application; an analyst must not infer an agent's internal reasons for stopping from the number of queries alone.

Second, record discovery and acquisition. Preserve queries, candidate sources, selected URLs, and read outcomes. Inspect official materials and record the community alternatives actually encountered, whether from mixed search results or subsequent searches. Keep a failed read separate from a failed search. When a mirror or another version supplies the required content, link it to the original attempt so that the workaround remains visible.

Third, assess task support and applicability. Map acquired material to the task requirements. Distinguish a returned document with missing essential content from a document that was never returned. Record stated versions and compatibility dependencies, including unresolved local facts. A version number in a URL is a clue, not by itself a verified compatibility relation.

Fourth, score from observations. Apply published anchors and explain the score assigned to each observation, and mark estimates and missing fields. Content acquisition status has four states: core content obtained; partial content obtained; required content not obtained; and unknown. Record access route separately as original entry, alternative entry, or unknown. Thus a complete mirrored body remains core content obtained even when it arrived through an alternative entry. Adequacy can use ordered anchors ranging from a generic fragment, through an overview and a

usable main path, to detailed task-relevant reference material. The coding rationale identifies the specific requirements covered; it should not rely only on page length.

Fifth, inspect the instructions and report the profile. Identify the response or guidance used to assess M9 and M10. Map its key steps and version claims to acquired evidence, and record contradictions, source dependencies, and unsupported steps. Report the composite-confidence formula, missing-data treatment, and calculation assumptions. The accompanying worked example connects requirements to events, sources, acquired evidence, and scores, with raw call counts shown alongside the M8 effort score.

The method's novelty lies in this connected specification of the unit, evidence record, indicator roles, and diagnostic interpretation. Search rank, reading success, and version labels are familiar observations. Their value here comes from relating them to the requirements of one development task and explaining how an observation becomes a diagnosis.

4.3 Scoring rules and composite confidence

Each indicator is mapped to an ordinal score from 1 to 5 using its scoring rule. We describe the anchors by knowledge source, acquisition effort, response properties, and composite confidence. Appendix A specifies decision priorities and adjustments; task records provide the corresponding observations.

Official material (M1–M4). Discoverability combines result position, search rounds, and query refinement; body accessibility uses the observed fetch category. Body detail combines task coverage with the actionability of commands and code, while source version clarity considers version identifiers and compatibility conditions. Figure 2 shows the four sets of anchors: the lowest M3 grades distinguish whether task-specific details are present, while M4 distinguishes missing identifiers from listed versions with unclear compatibility. M3 is marked blocked when its body cannot be acquired.

Score		1	2	3	4	5
M1	Official source discoverability	Still not found	Refine query / rank >10	Multiple rounds / rank 7–10	First round: 2–6	First round: 1
M2	Official content accessibility	robots restriction	SPA blocked	—	SSR / partial return	Static body
M3	Official content detail	No task details	A few task fragments	Overview / main path, no code	Main path + code / full ref., no code	Full reference + commands/code
M4	Source version clarity	Version IDs missing	≥3 versions listed pairing unclear	Chip–framework pairing / other	Support matrix	Version irrelevant / at most one

Fig. 2. Scoring anchors for official material (M1–M4); M2 uses four categories.

Third-party material, model knowledge, and effort (M5–M8). M5 scores the number of distinct third-party sources. M6 starts from the mean source-type baseline, adjusted for consistency, publication age, and platform distribution. M7 starts from the model self-rating and adjusts for technical change; Figure 3 shows the anchors before adjustment. M8 uses weighted effort $c = r + 0.5f + 4e$, where r, f, and e denote search rounds, fetch calls, and failed fetches.

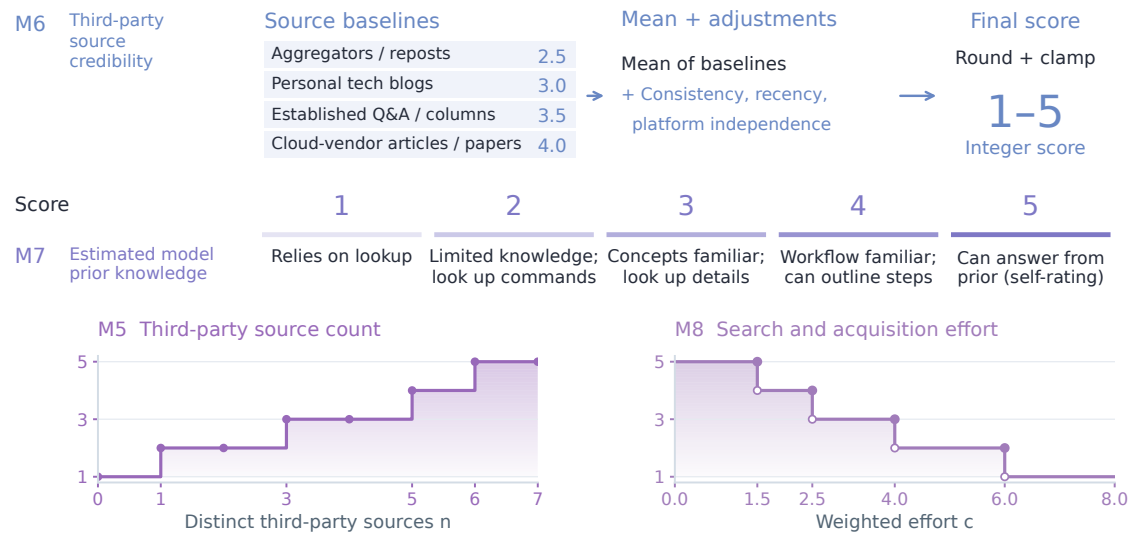

Fig. 3. Scoring components for M5–M8: M7 self-ratings, M5 and M8 mappings, and M6 source-type baselines with adjustments. Appendix A specifies adjustment values.

Response properties (M9–M10). M9 examines whether a response specifies versions and component combinations applicable to the task; M10 examines the operational conditions of its commands and steps. Figure 4 distinguishes the lower grades: version clues with unresolved applicability score above no specific version information, and a main workflow outline lacking key details scores above isolated fragments. These separately reported properties connect source conditions to the guidance produced.

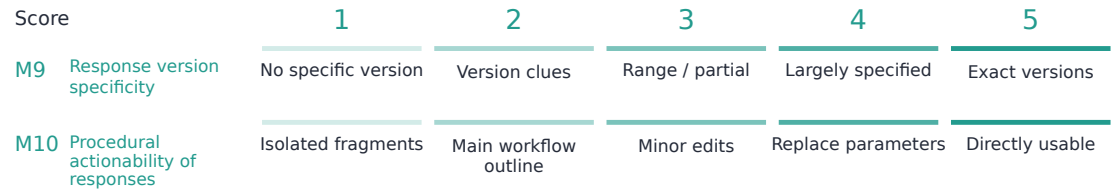

Fig. 4. Scoring anchors for answer version specificity (M9) and step actionability (M10).

Composite confidence (M11). Composite confidence is calculated from M1–M8. Each indicator score s_i is normalized using $n(s_i) = s_i/5$. When M3 is marked as blocked, its contribution to the official-source branch is set to zero:

$$O = n(s_1)n(s_2)n(s_3), \quad C = n(s_5)n(s_6), \quad P = n(s_7).$$

$$I = [1 - (1 - O)(1 - C)(1 - P)]\,[0.7 + 0.3n(s_4)]\,[0.9 + 0.1n(s_8)]\,.$$

The formula first combines support from official material, third-party material, and model prior knowledge, then applies version-clarity and retrieval-effort factors. It represents the potential for different knowledge channels to complement one another. Composite confidence is calculated from normalized ordinal scores and displayed using the bands in Figure 5; M9 and M10 are reported separately as response properties.

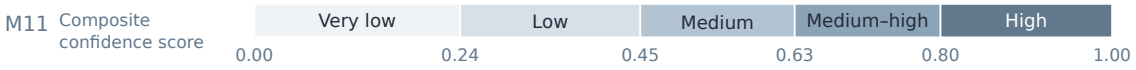

Fig. 5. Composite-confidence bands. Lower bounds are inclusive; the highest band includes 1.

The task matrix presents the indicator scores, while the access-profile figure shows the corresponding content-acquisition outcomes to support interpretation. Appendix A specifies the calculations; the supplement provides task-level worked examples.

5 CASE STUDY OF THE CANN AND CUDA DEVELOPER ECOSYSTEMS

5.1 Task coverage and comparison scope

We conducted the development-task audit on June 10-11, 2026, covering environment setup, operator development, training, inference and deployment, performance analysis, debugging, and migration. Examples include model conversion, custom-operator integration, distributed initialization, version compatibility, and memory-error investigation. The task set sought workflow coverage; it was not sampled to estimate how often developers encounter these needs. Developer-role groupings organize task scenarios around the typical knowledge needs of different roles.

CANN and CUDA provide a useful setting because the tasks involve specialized APIs, toolchains, framework integration, and version dependencies. The task wording uses each ecosystem's terminology. This yields contextual comparisons, not controlled substitutions of equivalent APIs. In task A, for example, the CUDA question begins with a PyTorch model and asks for a TensorRT deployment path, whereas the CANN question starts with an ONNX model and asks for an ATC conversion. These scope differences must remain visible when interpreting effort and completeness.

Task G is a distinct migration analogy. Its comparison question concerns moving CUDA code to AMD ROCm/HIP and its official material comes from AMD. It is included in the 26-task set as a worked migration case, but is excluded from CANN/CUDA aggregate comparisons, leaving 25 pairs and 50 task-side records for those summaries. Its separate treatment demonstrates why source and comparison identity belong in the measurement protocol.

5.2 Task execution and development of the method

We used a process log to record task questions, query strings, source descriptions, read outcomes, and scoring rationales. Scoring functions converted the observation fields into an interactive task matrix and analytic visualizations. Initial tasks used stepwise records; later batches used structured summaries. Some early read assessments reused observations already obtained in the same session.

Tasks were conducted in four batches. The first two covered A–D and E–H; the later two covered I–S and T–Z through parallel subagents. Structured observations were consolidated in the main session, where official and third-party source ownership was checked. Task questions were linked to their original assignments, and wording for the later batches was added to the process log from those assignments.

Tasks used Claude with web-search and web-reading tools. Search returned candidate sources, and the reading tool summarized HTML from selected URLs. The analysis combined recorded queries, tool-returned content and summaries, structured observations, and scoring rationales. M9 and M10 assessed version specificity and procedural actionability using the guidance produced during the tasks and its corresponding records. The supplement links tasks, sources, and scoring rationales.

The framework developed iteratively through the audit. A consequential revision was the rejection of a site-wide access assumption when a main-path quickstart returned useful content. This paper consolidates the protocol and separates observations from estimates and aggregation choices. The supplementary protocol and worked record make the resulting recording structure explicit.

5.3 Task-level assessment results

The main comparison includes 25 CANN tasks: core content was obtained in 22 tasks, partial content in two, and no core content in one. Core content was obtained in all 25 corresponding CUDA tasks. Figures 6–8 present all eleven indicator scores and group tasks by environment and installation, operator development, training, inference and deployment, performance and optimization, debugging, and migration. G is shown as a separately marked CANN/ROCm-HIP migration analogy and is excluded from CANN/CUDA summaries.

Figures 6–8 present the ordinal scores for M1–M10 and composite confidence calculated from M1–M8. Figure 9 adds content-acquisition states to support interpretation of the corresponding scores.

For the 25 task pairs, the composite confidence means are .732 for CANN and .922 for CUDA. The findings below examine task-level variation in composite confidence alongside the individual indicators. The supplementary analysis notes document source-ownership corrections, unresolved counting boundaries, and sensitivity to the model-prior term.

6 FINDINGS: KNOWLEDGE CONDITIONS ACROSS DEVELOPMENT TASKS

Reading the matrix across tasks and across indicators reveals four connected patterns. Access failures are concentrated in particular acquisition routes, while version ambiguity recurs across otherwise different tasks. Official, third-party, and model-prior assessments form different support profiles, and workflow position alone does not explain those profiles. We develop each finding through specific tasks and use the combination of indicators to locate its implications.

6.1 Acquisition breakdowns vary by task and access path

The agent obtained useful official content in CANN installation, conversion, training, and optimization tasks, with partial or unsuccessful acquisition along a small number of routes. This distribution supports inspection at the level of a task and its selected resources. A single site label, such as static or dynamically rendered, cannot describe the content obtained along every route through that site. Core official content was obtained in all 25 CUDA tasks, providing a contrast for these acquisition routes. Search rounds also differed: 13 of 25 CANN tasks and 20 of 25 CUDA tasks used one search round; the remainder used two. These counts distinguish search effort from whether the selected content was subsequently obtained.

Task D requires creating an Ascend C operator and integrating it with a framework, with implementation code, configuration-field explanations, and compilation and deployment steps. The WebFetch return for the selected official entry contained navigation, metadata, and documentation links, explicitly reporting “No code blocks, no implementation details, no step-by-step commands.” This read therefore did not supply the implementation content required by

A. Official-source conditions

Indicator scores by workflow. Each task row has CANN and CUDA columns; G uses ROCm/HIP in the second column.

Task	M1 Official find		M2 Official access		M3 Official detail		M4 Source version	
	CANN	CUDA‡	CANN	CUDA‡	CANN	CUDA‡	CANN	CUDA‡
Environment and installation								
I Version compatibility	3	3	4	5	4	5	4	4
J Installation	5	4	4	5	5	5	2	3
K Container setup	5	5	5	5	5	4	4	5
Operator development								
C Library selection	4	5	4	5	3	5	3	5
D Custom operator	2	5	2	5	—	5	2	3
L Operator accuracy	2	2	4	5	5	5	2	5
M Dynamic shapes / tiling	2	5	4	5	4	5	4	4
N Operator fusion	5	2	4	5	5	4	2	5
O Framework integration	2	5	4	5	5	5	3	3
Training								
F Distributed training	4	5	4	5	4	5	2	3
P Mixed precision	3	5	4	5	4	5	4	5
Q Memory / OOM	5	5	4	5	5	5	4	5
R Training convergence	3	3	4	5	5	4	3	5
Inference and deployment								
A Model conversion	4	5	4	5	4	5	2	4
H Quantization	3	5	4	5	3	5	2	3
S Inference serving	5	5	4	5	4	5	3	4
T Dynamic inference	3	3	4	5	5	5	2	5
Performance and optimization								
B Profiling	4	4	4	5	4	5	2	4
U Memory / occupancy	4	5	4	5	4	5	5	5
V Compute-transfer overlap	4	5	4	5	4	5	2	5
W Multi-device communication	2	5	4	5	3	4	3	4
Debugging								
E Error-code diagnosis	3	4	4	5	2	5	2	5
X Memory-error diagnosis	5	5	4	5	5	5	2	3
Migration								
Y Chip migration	2	5	4	5	4	5	3	4
Z Programming concepts	5	5	4	5	5	4	5	5
Migration analogy (not in CANN/CUDA summaries)								
G Migration analogy	4	5	4	5	4	5	4	3

M1–M10 are ordinal scores (1–5; higher is more favorable; M8 is an inverse effort score). "—" = unobserved official content detail.

* M7 estimates model prior knowledge. † M11 is composite confidence from M1–M8. ‡ For G, CUDA‡ = ROCm/HIP; it is excluded from CANN/CUDA summaries.

Fig. 6. Full indicator matrix (1/3): M1–M4 scores for official discoverability, official content accessibility, official content detail, and source version clarity, organized by workflow with CANN and CUDA columns for each task. G uses ROCm/HIP in the second column.

the task. M1 records official-source discovery conditions, M2 records the acquisition outcome, and M3 is marked as blocked. This assessment locates an improvement target in the path from the selected entry to the implementation body: the entry should expose a body link that the reading tool can follow, or directly present the implementation, compilation, and deployment instructions. For assessing an implementation suggestion, the missing information is the procedure that would support its steps; a link to the guide alone does not supply that basis.

Task E exposes a different problem. The official EZ9999 page was found and read, but its explanation supplied an unspecified cause and generic instructions to check the installation or inspect logs. Its low content-detail code reflects the lack of concrete diagnostic guidance in the returned material. The developer's question requires information about possible triggers and a sequence for narrowing the cause. Improving extraction would leave this content problem in place. Together, D and E connect similar concerns about obtaining useful guidance to distinct interventions: delivery of the missing how-to body and enrichment of an accessible error explanation. The corresponding need is evidence for

B. Third-party, prior, and acquisition conditions

Indicator scores by workflow. Each task row has CANN and CUDA columns; G uses ROCm/HIP in the second column.

Task		M5 3P count		M6 3P credibility		M7 Prior estimate*		M8 Search / access effort	
		CANN	CUDA‡	CANN	CUDA‡	CANN	CUDA‡	CANN	CUDA‡
Environment and installation									
I	Version compatibility	3	2	4	4	2	4	3	3
J	Installation	3	3	4	4	4	5	5	5
K	Container setup	2	3	2	4	3	5	1	4
Operator development									
C	Library selection	3	5	4	4	3	5	5	5
D	Custom operator	3	4	2	4	2	5	1	5
L	Operator accuracy	3	2	4	4	3	4	4	1
M	Dynamic shapes / tiling	2	2	4	4	2	5	3	5
N	Operator fusion	2	2	3	4	2	4	5	1
O	Framework integration	3	2	5	4	3	5	1	3
Training									
F	Distributed training	3	3	3	4	3	4	5	5
P	Mixed precision	3	3	4	4	4	5	2	4
Q	Memory / OOM	3	3	4	4	3	5	3	2
R	Training convergence	3	3	4	3	3	5	3	4
Inference and deployment									
A	Model conversion	4	4	4	4	3	4	4	5
H	Quantization	4	3	3	4	3	4	4	5
S	Inference serving	2	3	4	4	2	4	4	5
T	Dynamic inference	3	2	5	4	3	5	3	1
Performance and optimization									
B	Profiling	3	4	3	4	3	4	4	5
U	Memory / occupancy	3	3	4	4	3	5	4	5
V	Compute-transfer overlap	3	3	4	4	3	5	4	4
W	Multi-device communication	3	3	4	4	3	4	3	5
Debugging									
E	Error-code diagnosis	3	4	3	4	2	4	4	5
X	Memory-error diagnosis	2	3	4	4	2	4	4	5
Migration									
Y	Chip migration	3	3	4	4	3	5	1	4
Z	Programming concepts	3	3	4	4	4	5	1	1
Migration analogy (not in CANN/CUDA summaries)									
G	Migration analogy	3	3	4	4	3	4	5	5

M1–M10 are ordinal scores (1–5; higher is more favorable; M8 is an inverse effort score). "—" = unobserved official content detail.

* M7 estimates model prior knowledge. † M11 is composite confidence from M1–M8. ‡ For G, CUDA‡ = ROCm/HIP; it is excluded from CANN/CUDA summaries.

Fig. 7. Full indicator matrix (2/3): M5–M8 scores for third-party source count, third-party source credibility, estimated model prior knowledge, and search and acquisition effort, in the same workflow and task order.

choosing a diagnostic check. Local logs may help distinguish possible causes, while documentation or attributed cases must explain how those observations relate to the proposed checks.

Task A shows how useful main-path material and unavailable references can coexist. The acquired conversion quickstart supplied an ATC command, input-shape and target-device parameters, and a device-information step. A more extensive ATC/AIPP reference returned navigation and metadata. The worked recording example links the quickstart to supported conversion requirements and the missing reference to unresolved advanced settings. In task Y, accessing chip-migration information through the document center likewise yielded only partial content. These cases motivate recording both the content obtained and its relation to a task requirement, with the access route recorded separately where available. For a developer assessing the conversion guidance, this distinction identifies a supported main procedure and advanced settings that still require reference material. It bounds which parts of a proposed action the acquired sources can substantiate.

C. Response checks and composite confidence

Indicator scores by workflow. Each task row has CANN and CUDA columns; G uses ROCm/HIP in the second column.

Task		M9 Response version		M10 Actionability		M11 Confidence†	
		CANN	CUDA‡	CANN	CUDA‡	CANN	CUDA‡
Environment and installation							
I	Version compatibility	5	5	5	5	0.73	0.85
J	Installation	4	4	5	5	0.80	0.88
K	Container setup	4	4	5	5	0.86	0.98
Operator development							
C	Library selection	4	5	4	5	0.77	1.00
D	Custom operator	2	5	3	5	0.41	0.88
L	Operator accuracy	3	4	4	4	0.69	0.84
M	Dynamic shapes / tiling	4	4	4	5	0.63	0.94
N	Operator fusion	4	4	4	4	0.74	0.83
O	Framework integration	4	4	5	5	0.72	0.84
Training							
F	Distributed training	3	4	3	5	0.72	0.88
P	Mixed precision	4	5	4	5	0.83	0.98
Q	Memory / OOM	3	4	4	5	0.86	0.94
R	Training convergence	3	4	4	4	0.75	0.98
Inference and deployment							
A	Model conversion	3	5	4	5	0.75	0.94
H	Quantization	4	4	5	5	0.69	0.88
S	Inference serving	4	4	4	5	0.74	0.94
T	Dynamic inference	4	5	4	5	0.72	0.92
Performance and optimization							
B	Profiling	3	4	3	5	0.70	0.93
U	Memory / occupancy	4	4	4	4	0.88	1.00
V	Compute-transfer overlap	4	5	4	5	0.72	0.98
W	Multi-device communication	4	5	4	5	0.70	0.92
Debugging							
E	Error-code diagnosis	3	4	3	5	0.55	0.99
X	Memory-error diagnosis	4	4	5	5	0.74	0.88
Migration							
Y	Chip migration	4	5	4	5	0.68	0.92
Z	Programming concepts	2	2	2	2	0.90	0.92
Migration analogy (not in CANN/CUDA summaries)							
G	Migration analogy	4	4	4	5	0.84	0.88

M1–M10 are ordinal scores (1–5; higher is more favorable; M8 is an inverse effort score). "—" = unobserved official content detail.

* M7 estimates model prior knowledge. † M11 is composite confidence from M1–M8. ‡ For G, CUDA‡ = ROCm/HIP; it is excluded from CANN/CUDA summaries.

Fig. 8. Full indicator matrix (3/3): M9–M11 values for response version specificity, procedural actionability of responses, and the composite confidence score from M1–M8, in the same workflow and task order.

6.2 Version applicability is a recurring cross-task issue

Version clarity is a recurrent limitation even where the official content is accessible. Twelve of the 25 CANN tasks have M4 = 2, spanning conversion, installation, training, operator development, inference, optimization, and debugging. Only U and Z receive M4 = 5, and both are coded as version-insensitive. M4 scores for CUDA tasks range from 3 to 5. Because the scoring rule uses version counts and compatibility-matrix availability, the observed distribution directs attention to the underlying version evidence rather than establishing ambiguity from version counts alone.

The tasks illustrate how this version information affects knowledge applicability. In A, similar quickstart material appears in more than one version tree. In I, answering a compatibility question requires connecting toolkit, driver, framework, and integration-package information across documentation, repositories, and package records. A page may be detailed and readable while still leaving the reader to determine which combination its instructions support. Version information therefore needs to accompany the content and connect to the environment of the task.

Access-profile detail

Content-acquisition states complement the M2 accessibility scores in Figures 2–4.

Task	Content access		Official content detail		Source version clarity	
	CANN	CUDA‡	CANN	CUDA‡	CANN	CUDA‡
Environment and installation						
I Version compatibility	C	C	4	5	4	4
J Installation	C	C	5	5	2	3
K Container setup	C	C	5	4	4	5
Operator development						
C Library selection	P	C	3	5	3	5
D Custom operator	N	C	—	5	2	3
L Operator accuracy	C	C	5	5	2	5
M Dynamic shapes / tiling	C	C	4	5	4	4
N Operator fusion	C	C	5	4	2	5
O Framework integration	C	C	5	5	3	3
Training						
F Distributed training	C	C	4	5	2	3
P Mixed precision	C	C	4	5	4	5
Q Memory / OOM	C	C	5	5	4	5
R Training convergence	C	C	5	4	3	5
Inference and deployment						
A Model conversion	C	C	4	5	2	4
H Quantization	C	C	3	5	2	3
S Inference serving	C	C	4	5	3	4
T Dynamic inference	C	C	5	5	2	5
Performance and optimization						
B Profiling	C	C	4	5	2	4
U Memory / occupancy	C	C	4	5	5	5
V Compute-transfer overlap	C	C	4	5	2	5
W Multi-device communication	C	C	3	4	3	4
Debugging						
E Error-code diagnosis	C	C	2	5	2	5
X Memory-error diagnosis	C	C	5	5	2	3
Migration						
Y Chip migration	P	C	4	5	3	4
Z Programming concepts	C	C	5	4	5	5
Migration analogy (not in CANN/CUDA summaries)						
G Migration analogy [CANN / ROCm-HIP]	C	C	4	5	4	3

Content status: C = core obtained; P = partial content obtained; N = not obtained. Access route is recorded separately in the protocol.

‡ For G, CUDA‡ = ROCm/HIP; it is a migration analogy and is excluded from CANN/CUDA summaries.

Fig. 9. Access-profile detail placing content-acquisition states beside official-content-detail and source-version-clarity scores. It follows the workflow groups and task order of Figures 6–8 to support interpretation of acquisition outcomes.

M4 and M9 make two stages of this problem visible. M4 describes version clarity in the source environment; M9 records how specifically the resulting guidance identifies an applicable version or combination. Reading them together helps locate whether the next improvement belongs in the documentation's compatibility information or in how the agent formulates the response. This recurring issue has a wider task footprint than the missing official body in D, although the local acquisition failure remains consequential for that task. For delegated retrieval, a further judgment concerns whether the source's stated conditions match the developer's device and software combination. Source applicability and local environment facts supply different parts of that comparison: a missing compatibility relation calls for source investigation, whereas an unspecified installed version calls for a local check.

6.3 Knowledge-support profiles differ across tasks

An accessible official guide can coexist with limited alternative material. In X, memory-error investigation, the acquired sanitizer documentation receives M3 = 5, while only one third-party source was found and M7 = 2. D also has a low model-prior estimate and weak third-party credibility, but lacks the core official body. The distinction is the available support configuration: X has detailed official guidance, whereas D combines a missing core guide with limited support in the other channels. A single label such as a thin community would obscure this difference.

Across the main comparison, retrieved third-party candidates average 3.16 per CANN task and 3.44 per CUDA task after the known vendor-blog exclusion in H. Counts alone provide a modest contrast and do not establish informational independence. Closer inspection of the search results revealed that some candidates came from the same platform, while some third-party pages could not be read beyond their search snippets. The method preserves source identity, acquisition outcome, and credibility assessment so that multiple entries are not automatically interpreted as multiple usable explanations.

The model-prior column adds a separate estimate of support. The CUDA scores are all 4 or 5; CANN estimates are frequently lower, including 2 for D, E, I, M, N, S, and X. These tasks combine lower prior estimates with needs for ecosystem-specific commands, compatibility relations, or diagnostic examples. They motivate examining where retrieved sources must carry more of the task's support. The estimates describe the audit's assessment of available model knowledge; the source observations identify the external material actually obtained.

M9 and M10 complete the profile by assessing the version specificity and actionability of guidance produced during the tasks. For example, X combines detailed official material with a higher procedural-actionability code than D. This association gives an analyst a concrete question to inspect: which steps can be supported by the acquired guide, and which still need information from another source or the local environment? The composite confidence score summarizes M1-M8, while the response scores are reported separately to examine the resulting guidance.

6.4 Task depth alone does not explain the observed differences

Workflow grouping helps locate concentrations of lower-scored tasks. Table 3 summarizes the 25-pair comparison. Operator development and debugging have lower CANN means than environment and installation, and debugging has the largest difference between the two columns. The debugging group consists of E and X, whose distinct profiles illustrate why a workflow mean needs to be read alongside its constituent tasks. The table describes the selected task set under the scoring rules used in this study.

Table 3. Workflow means of the composite confidence score for 25 task pairs. Difference = CUDA minus CANN; G is excluded.

Workflow	Tasks	CANN	CUDA	Difference
Environment and installation	3	0.799	0.904	0.106
Operator development	6	0.660	0.891	0.230
Training	4	0.791	0.945	0.154
Inference and deployment	4	0.722	0.920	0.198
Performance and optimization	4	0.752	0.957	0.205
Debugging	2	0.646	0.933	0.287
Migration	2	0.793	0.921	0.128

The broader distribution resembles an onboarding-to-specialization gradient, but contrasting cases qualify that interpretation. Installation and container setup have relatively high CANN composite scores, while custom-operator development and dynamic-shape/tiling tasks score lower. Yet U, a technically demanding memory/occupancy task, scores .881, and Z, programming-concept comparison, scores .901. Both involve principles that can be expressed across architectures and have acquired official material. E is comparatively straightforward as a question but depends on a specific error's diagnostic content and scores .554. Complexity or workflow position alone therefore misses important differences in knowledge requirements.

Dependence on ecosystem-specific knowledge provides an interpretive lens for these cases. General concepts of memory management can draw on cross-platform explanations; a vendor-specific error, operator interface, or compatibility combination requires information tied to that ecosystem. This lens directs the audit toward the knowledge that must be supplied explicitly for a task, rather than assigning a uniform expectation to all advanced work. It also gives developer-role groupings a practical use: grouping the tasks associated with a role can identify which specific knowledge requirements need attention. The present cases motivate this interpretation; testing its predictive value would require an explicit dependency coding scheme and a broader task sample.

7 DISCUSSION: DESIGN IMPLICATIONS FOR KNOWLEDGE PROVISION AND AGENT INTERACTION

The findings identify knowledge conditions on which developers' assessment of AI-provided guidance can depend. We discuss two sites of potential improvement: technical knowledge supplied by documentation and communities, and the agent interfaces through which that knowledge is presented. The proposals below follow from the observed task profiles. Figures 10–15 illustrate three supply-side and three agent-side implications: compatibility relations, diagnostic content, readable exports, acquisition feedback, environment checking, and source inspection. They illustrate possible applications of the diagnosis rather than evaluated interventions.

7.1 Supply-side design: version relations, task guidance, and alternative sources

Recurring version issues suggest treating applicability as a property of a documentation system. A stable recommended-version entry can coexist with clearly indexed documentation for earlier versions, while each page states its applicable versions and verification date. Compatibility relations among drivers, toolkits, frameworks, and integration packages can be published in a queryable form. For a task such as I, this would let a developer or agent request the supported combinations for a stated environment, reducing the need to assemble the relation from several disconnected records.

Figure 10 makes this proposal concrete through a compatibility lookup interface. Device and framework constraints define the query, and each returned combination retains its sources, applicability scope, and verification date. Where no supported match is documented, the interface identifies unresolved conditions. The version problem registered by M4 becomes an inspectable relation, providing a basis for subsequently checking the applicability of a response.

Localized acquisition and content gaps call for task-specific repairs. D motivates a readable route to the core operator-development guide, through server-rendered content, a text export, or another stable entry. E already has a dedicated error page; its improvement requires more informative content within that entry: error semantics, possible triggers, diagnostic steps, links to issue cases, and applicable versions. For a catch-all error, a guide can state what the code alone leaves unresolved and specify which logs or local observations are needed next. Such a page supports diagnosis by organizing available evidence around the user's problem.

Figure 11 illustrates task-oriented content through E's error page. It starts with what the code can establish, identifies local information to collect, links observable symptoms to checks and attributed cases, and retains a support route for

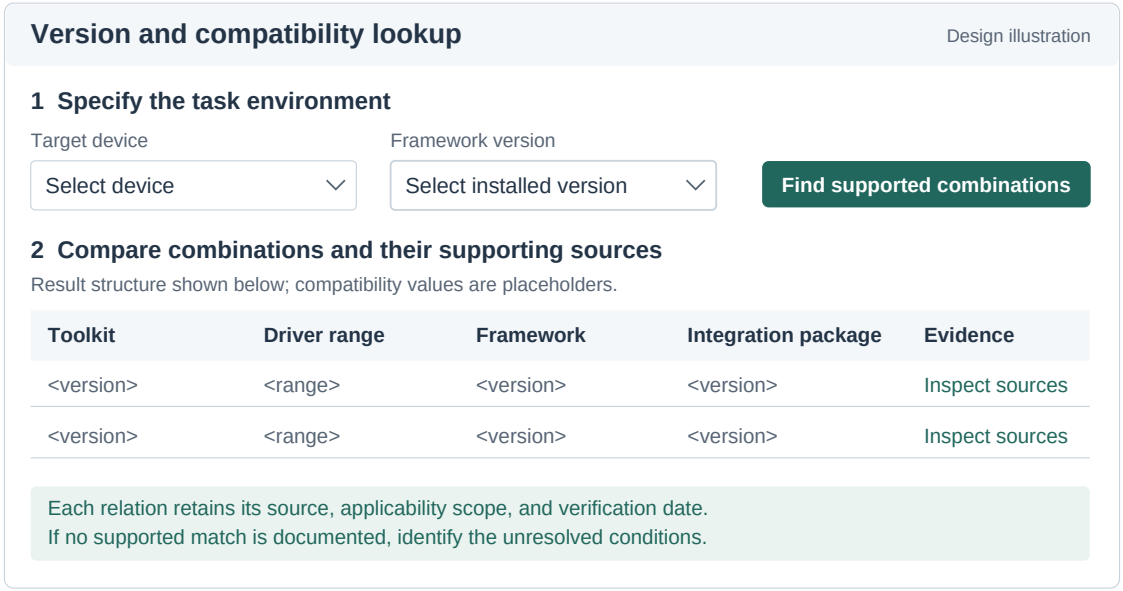

Fig. 10. Proposed version and compatibility lookup, motivated by fragmented version information in A and I. Query constraints, result fields, and source inspection illustrate the proposed information structure; compatibility values are placeholders.

unresolved situations. Domain maintainers would need to establish the actual causes, checks, and cases; the structure helps locate the knowledge that needs to be supplied.

The same task-oriented structure can guide other documentation units. An intended outcome, prerequisites, minimal commands or code, parameter explanations, expected output, and recovery information provide concrete material for both human reading and agent retrieval. Machine-readable exports and indexes should preserve those relationships and their version scope. Their success can be checked against the relevant content-acquisition and detail fields for the task that motivated the change.

Figure 12 illustrates delivery of the same task material through a document body and a machine-readable export. Using the command and parameter relations in the conversion quickstart acquired in task A, the export preserves the task, applicable version, source, parameter explanations, and related steps. For a route such as D's, the aim is to deliver the missing core guidance; where a readable body already exists, the aim is to preserve its context during export. Acceptance checks should compare acquired content with task requirements, rather than only checking for a particular file format.

Third-party sources offer additional examples, explanations, and records of encountered failures. Their value depends on what an agent can actually obtain and attribute. Supporting accessible, versioned examples and accounts of issue resolution can extend the available knowledge beyond an official main path. M5 and M6 allow an audit to distinguish the number of encountered sources from their credibility and derivation. This is particularly useful when several posts repeat the same material or when a detailed explanation is visible in search but unavailable to the reading tool.

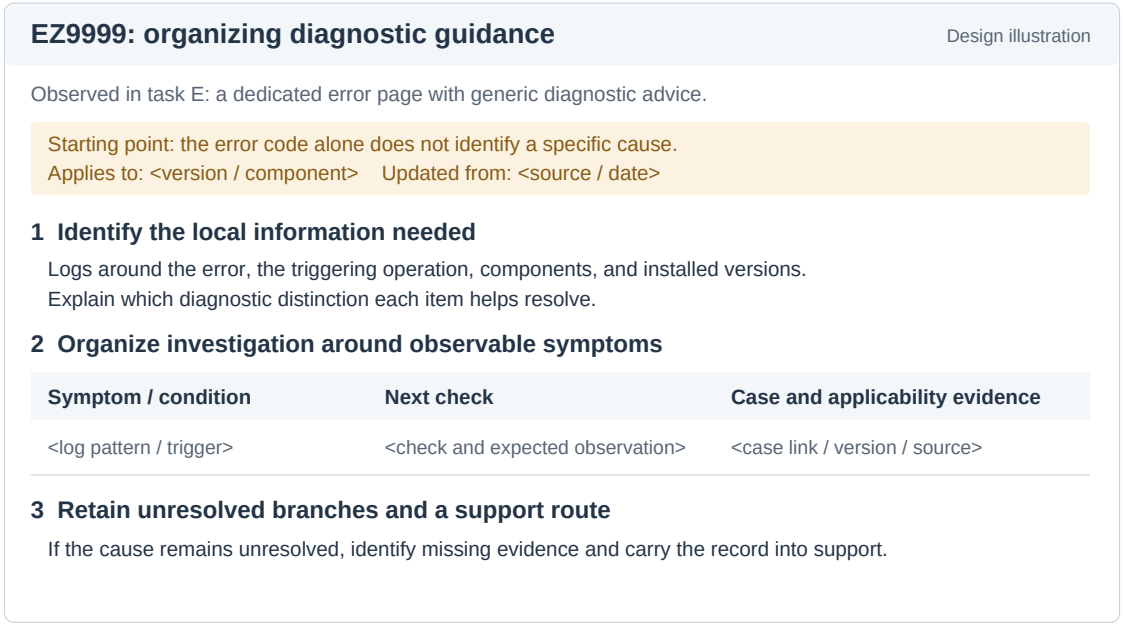

Fig. 11. Proposed task-oriented diagnostic content, motivated by E's dedicated error page with generic guidance. The structure connects a diagnostic starting point, local information, checks, case evidence, and unresolved branches. Specific cases and version scopes remain fields to populate.

Knowledge provision also involves entry points and maintenance. Stable, searchable official entries affect whether candidate materials are discovered; explicit ownership, revision dates, and compatibility relations support subsequent assessment. Teams can use representative tasks as recurring inspection units, tracking whether queries reach the intended material, its body is delivered, version relations have changed, and community cases remain applicable. Such maintenance also addresses source governance and third-party knowledge provision beyond the illustrated interfaces.

7.2 Agent-side design: expose support and request missing task facts

An agent interface can preserve the difference between finding a relevant source, obtaining its body, and supporting a proposed step. For D, an informative status would identify the missing implementation guide. For E, it would indicate that the official explanation is generic and request the logs or trigger context needed for further diagnosis. For A, it could retain the usable conversion path while marking advanced reference settings as unresolved. These statuses connect a retrieval event to its consequence for the developer's current task.

Figure 13 contrasts proposed responses to D and E. For D, the agent first seeks a linked body page or another readable source, while retaining the original entry for optional inspection. For E, it explains how trigger context and relevant log excerpts would guide a search for diagnostic cases. Developer-supplied content is a fallback for acquisition failures; accessible core guidance remains a responsibility of the knowledge provider. M1-M3 locate the gap, while M8 records the effort of subsequent attempts.

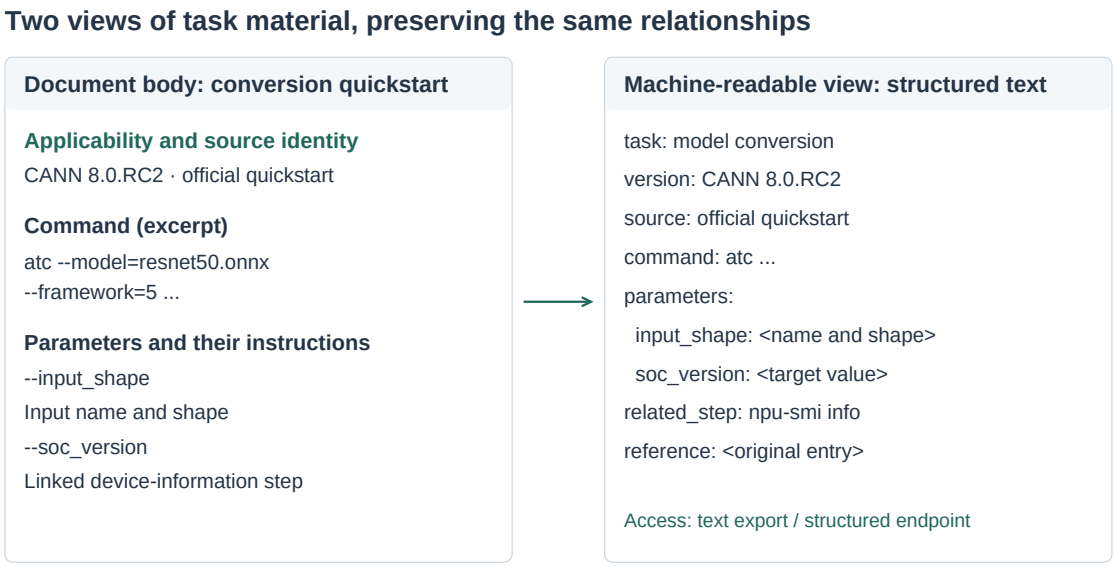

Fig. 12. Proposed document body and machine-readable export. Command, parameter, and device-query relations come from the conversion quickstart acquired in task A; the structured text is a proposed export. Both views preserve the task's version, source, and step relationships.

Version-dependent guidance should connect a source's applicability to the available task environment. Before proposing a toolkit/framework combination, an agent can use device and software facts already supplied in the task, requesting only information that remains necessary to establish the match. The response indicators then ask whether the recommendation states its applicable version conditions and provides actionable steps with explicit substitutions or modifications.

Figure 14 illustrates version checking through a separate environment record. It distinguishes developer-supplied local facts from the applicability declared by a source, then records the comparison as matched, mismatched, or insufficient information. The record is reused within the task and updated with additional user input to support applicability checks before specific commands are recommended.

The three knowledge channels also suggest useful distinctions in how an agent presents support. A cited official procedure, a community workaround, and an explanation based on model prior knowledge have different provenance. Showing those differences can help a developer identify which part of a response warrants a local check. The retrieval loop can direct further searches toward a missing requirement and report remaining uncertainty when its budget is reached. This connects source inspection to collaboration through observable information needs and verification points.

Figure 15 further illustrates source inspection and corrective feedback. A developer can expand the cited passage for a command, inspect its applicable version, and attach an encountered error or correction to the suggestion. The original source, task environment, and new feedback remain connected so the next check can address the specific discrepancy. This provides inspectable support for the response properties described by M9 and M10 and for subsequent revision.

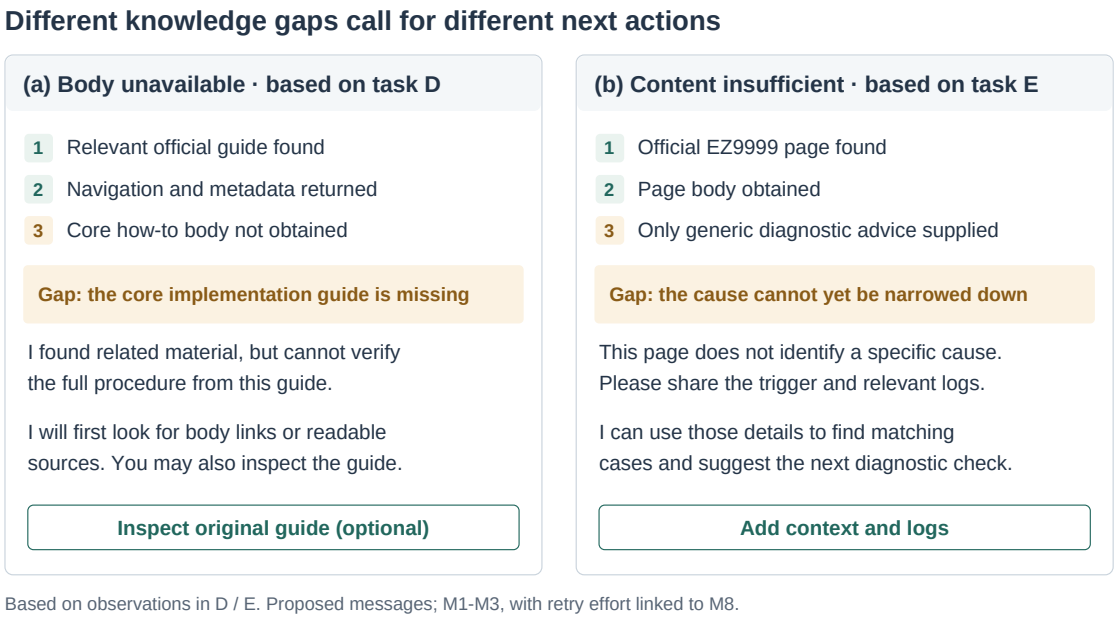

Fig. 13. Proposed agent feedback for two knowledge gaps: (a) a missing core body, based on D; (b) readable but diagnostically insufficient content, based on E. Task observations supply the problem conditions; interface messages and next actions are design proposals.

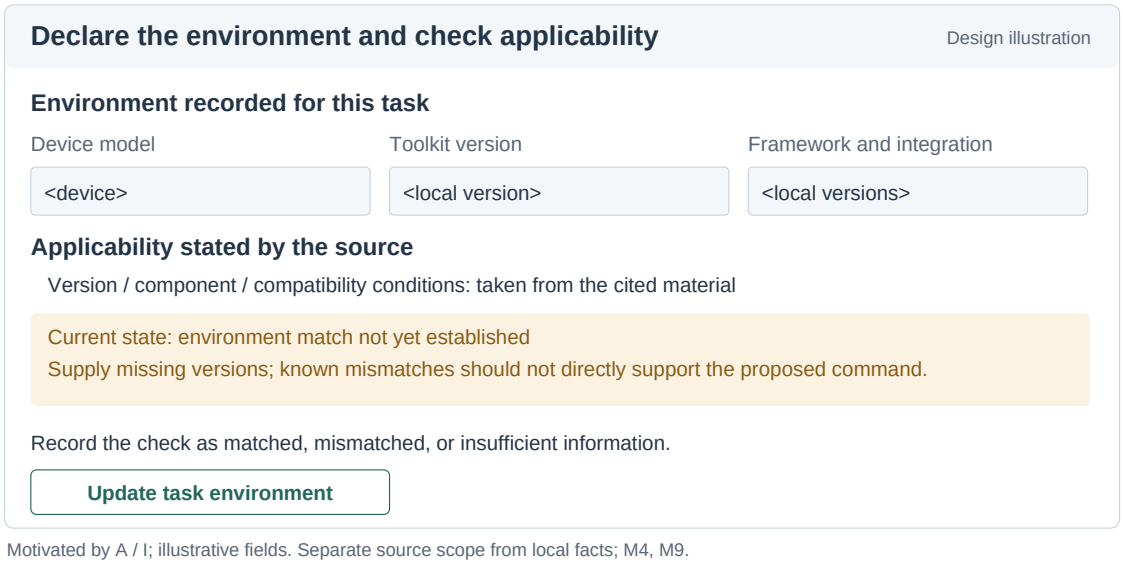

Fig. 14. Proposed environment declaration and version checking, motivated by A and I. Device, toolkit, and framework fields represent developer-supplied local facts, separate from source applicability. The illustrated state has insufficient information to establish a match.

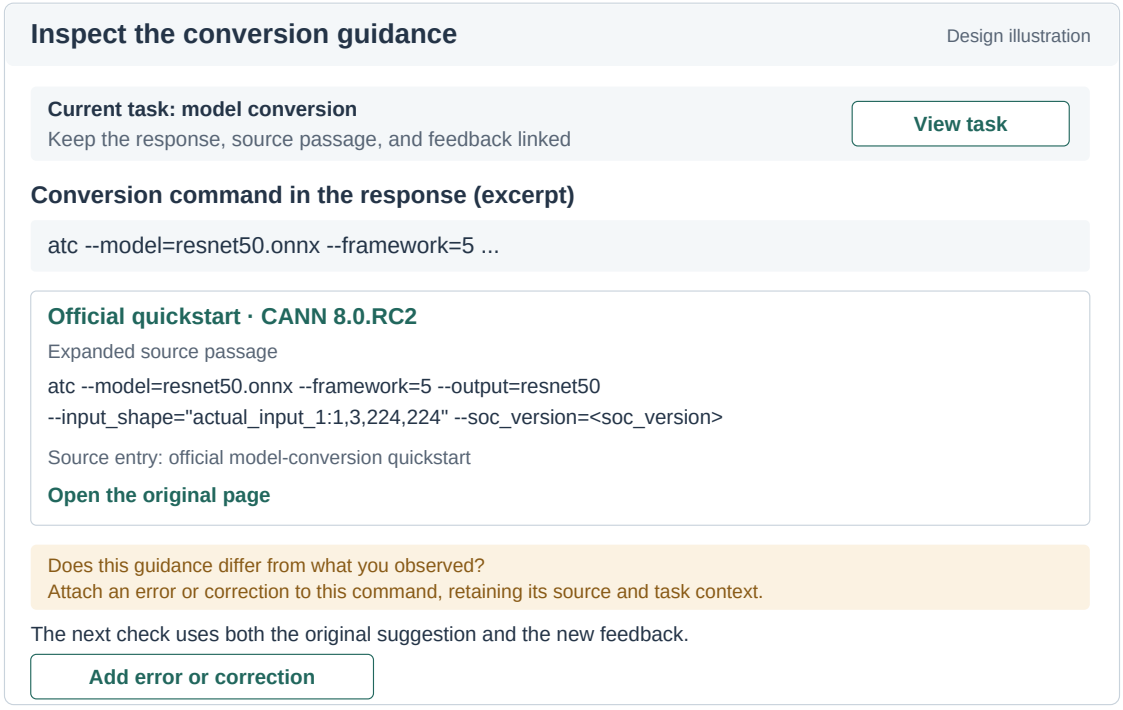

Fig. 15. Proposed source inspection and corrective feedback. The conversion-command excerpt comes from the quickstart acquired in task A; source expansion, an original-page entry, and error feedback linked to the suggestion are proposed interactions.

These proposals treat checking suggestions as part of programming work, consistent with the information-seeking and AI-assistance studies discussed earlier [1, 2, 13]. Exposing every retrieval event or source passage could add reading effort without clarifying the next action. An interface could first show the supported step, relevant version scope, and unresolved condition, with source passages available on demand. This prioritizes the information needed for the current judgment while retaining a route to detailed inspection.

Requests for environment facts can likewise interrupt a task, especially when they repeat information already provided. A request should explain which recommendation depends on the missing fact and reuse task context where available, with confirmation when it may be stale. For diagnostic logs, it should request relevant excerpts and allow sensitive details to be removed. Missing compatibility relations and generic error explanations require work by documentation and system teams; collecting more local facts cannot supply those missing explanations. The balance between inspection, interruption, and useful guidance remains a question for evaluating these proposals.

7.3 Prioritizing widespread improvements and localized repairs

The audit supports two complementary ways to organize improvement work. One considers how widely a condition recurs: version ambiguity across several workflows motivates a shared compatibility capability. The other considers the severity and specificity of a breakdown: a missing operator guide or uninformative error page motivates repair

of a particular task path. These scopes imply different units of progress. Broad improvements can be tracked by the number of affected tasks whose applicability information becomes clearer; local repairs can be tracked by whether the previously missing body or diagnostic content becomes available.

A mean score alone favors changes that touch many tasks and can make a repair to one severe gap appear small. Conversely, focusing only on the lowest-scored task would leave recurring issues elsewhere unaddressed. The full profile allows both concerns to remain visible. Composite confidence can provide a summary within the declared rules, while task-level observations identify the intervention target. Formula sensitivity and calculation notes are supplied in the supplement to support inspection of those summaries.

7.4 Reusing the diagnostic method

The method connects a task requirement to a source or acquisition event, an indicator code, and a diagnosis of the information needed to assess a proposed action. Documentation maintainers could use the profile to locate missing guidance or applicability relations; agent product and interface teams could use it to distinguish further source retrieval from a request for a local fact. Analysts can reuse the record structure with domain-specific requirements, such as client-server compatibility in a database SDK or deployment prerequisites in a web framework. These are intended applications of the method.

Subsequent evaluation can examine whether independent analysts produce comparable profiles, whether the diagnoses correspond to expert assessments of missing information, and whether documentation teams find them useful for planning repairs. These questions follow directly from the intended use of the method. The present application supplies the task-level distinctions, worked records, and cross-task synthesis on which such evaluation can build.

8 LIMITATIONS AND RESEARCH TRANSPARENCY

The case demonstrates how the method distinguishes breakdowns in discovery, content acquisition, and task adequacy. Task records support inspection of scoring rationales and reproduction of the calculations. The results reflect this task set and its retrieval conditions. Stability across models, tool configurations, and retrieval budgets, as well as independent scoring agreement and applicability across ecosystems, requires further evaluation. The supplement provides observation fields, the scoring implementation, task-to-source mappings, calculation discrepancy notes, and summary scripts to support inspection and subsequent applications.

The study assesses knowledge conditions and recorded guidance. How developers use these distinctions to evaluate evidence, check local applicability, and decide when further information is needed requires observation of their interaction with that guidance. Evaluation should also examine whether the proposed feedback supports those judgments and what reading effort or interruptions it introduces.

9 CONCLUSION

We define knowledge availability for AI and operationalize it through eleven indicators connecting knowledge sources, acquisition processes, and response properties. The case study of the CANN and CUDA developer ecosystems shows how this multidimensional view identifies distinct acquisition and content failures, recurring version issues, and task-dependent configurations of knowledge support. Contrasting tasks motivate attention to ecosystem-specific knowledge requirements alongside workflow coverage.

The method makes the basis for AI-mediated technical guidance an object of inspection: which source supports a proposed step, which applicability relation remains unresolved, and what additional information is needed. The framework, protocol, and case analysis provide a way to locate these conditions and relate them to knowledge provision and agent feedback. Documentation repairs, source retrieval, and requests for local facts follow from different diagnoses, offering concrete directions for subsequent evaluation.

APPENDIX A: SCORING CALCULATIONS

Official material (M1–M4). M1 first considers the search process: at least two rounds with query refinement scores 2; at least two rounds without refinement scores 3. For a single round, the first official result at rank 1, 2–6, 7–10, or beyond 10 scores 5, 4, 3, or 2, respectively. Grade 1 describes difficulty finding a relevant official source after repeated searches. M2 uses the four acquisition categories in Figure 2. M3 is blocked when SPA or robots barriers prevent body acquisition. For acquired content, grade 1 provides no task-specific details, grade 2 only a few task-specific fragments, and grade 3 an overview or a main path without commands/code. A main path with commands/code or a complete reference without them scores 4; a complete reference with them scores 5. Commands/code denotes operational content in the material. For M4, grade 1 lacks version identifiers; grade 2 lists at least three versions but leaves their compatibility unclear. Where version information is assessable, apply these conditions in order: version-irrelevant task or at most one version, 5; official support matrix, 4; identifiable device–framework pairing, 3; at least three versions with unclear compatibility, 2; remaining cases, 3.

Third-party material (M5–M6). For M5, distinct source counts n of 0, 1–2, 3–4, 5, and at least 6 map to grades 1–5. Official documents, repositories, and forums belong to the official channel. M6 starts with the mean source-type baseline: aggregators or reposts, 2.5; personal technical blogs, 3.0; established Q&A or columns, 3.5; cloud-vendor technical articles or academic papers, 4.0. Three adjustments follow. High, medium, and low content consistency add 1, 0, and −1. Relative to June 2026, median publication ages of at most 36 months, over 36 through 48 months, and over 48 months add 0, −0.25, and −0.5. Platform independence p is distinct platforms divided by source count; $p \geq 0.8$, $0.6 \leq p < 0.8$, and $p < 0.6$ add 0, −0.25, and −0.5. Missing dates are excluded from the median; an entirely missing date or platform field yields zero for that adjustment. Round the result to the nearest integer, resolving half-integer ties to the even integer, then clamp to 1–5.

Model knowledge and acquisition effort (M7–M8). M7 starts from the self-rating anchors in Figure 3. Stable, moderate, and fast technical change add 0, −0.25, and −0.5, respectively; rounding and clamping follow M6. M8 uses weighted effort $c = r + 0.5f + 4e$, where r is search rounds, f fetch calls, and e failed fetch calls. The intervals $c \leq 1.5$, $1.5 < c \leq 2.5$, $2.5 < c \leq 4$, $4 < c \leq 6$, and $c > 6$ map to grades 5, 4, 3, 2, and 1.

Response properties (M9–M10). M9 grade 1 provides no specific version; grade 2 supplies version clues but leaves applicability to the task unresolved; grade 3 specifies a range or some components; grade 4 largely specifies versions; grade 5 gives exact versions. M10 grade 1 offers isolated fragments; grade 2 outlines the main workflow but lacks key implementation details; grade 3 requires minor edits; grade 4 only parameter substitution; grade 5 is directly usable.

Composite confidence (M11). The main-text formula combines official, third-party, and model-knowledge branches, adjusted by M4 and M8. A blocked M3 sets the official branch $O = 0$. M9 and M10 remain separate. The five bands in Figure 5 have lower bounds of 0, 0.24, 0.45, 0.63, and 0.80. Each lower bound is inclusive; each upper bound is exclusive, except that the highest band includes 1.

REFERENCES

[1] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111. https://doi.org/10.1145/3586030

[2] Joel Brandt, Philip J. Guo, Joel Lewenstein, Mira Dontcheva, and Scott R. Klemmer. 2009. Two Studies of Opportunistic Programming: Interleaving Web Foraging, Learning, and Writing Code. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1589–1598. https://doi.org/10.1145/1518701.1518944

[3] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, 150–158. https://doi.org/10.18653/v1/2024.eacl-demo.16

[4] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. 54107–54157. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[6] Amy J. Ko, Robert DeLine, and Gina Venolia. 2007. Information Needs in Collocated Software Development Teams. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 344–353. https://doi.org/10.1109/ICSE.2007.45

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[8] Peter Pirolli and Stuart Card. 1999. Information Foraging. *Psychological Review* 106, 4 (1999), 643–675. https://doi.org/10.1037/0033-295X.106.4.643

[9] Martin P. Robillard. 2009. What Makes APIs Hard to Learn? Answers from Developers. *IEEE Software* 26, 6 (2009), 27–34. https://doi.org/10.1109/MS.2009.193

[10] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. Association for Computing Machinery, 491–514. https://doi.org/10.1145/3581641.3584037

[11] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 338–354. https://doi.org/10.18653/v1/2024.naacl-long.20

[12] J. Sillito, G. C. Murphy, and K. De Volder. 2008. Asking and Answering Questions during a Programming Change Task. *IEEE Transactions on Software Engineering* 34, 4 (2008), 434–451. https://doi.org/10.1109/TSE.2008.26

[13] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. Association for Computing Machinery, 1–7. https://doi.org/10.1145/3491101.3519665

[14] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. 50528–50652. https://doi.org/10.52202/079017-1601

[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629v3

[16] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *The Twelfth International Conference on Learning Representations*. 15585–15606. https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html